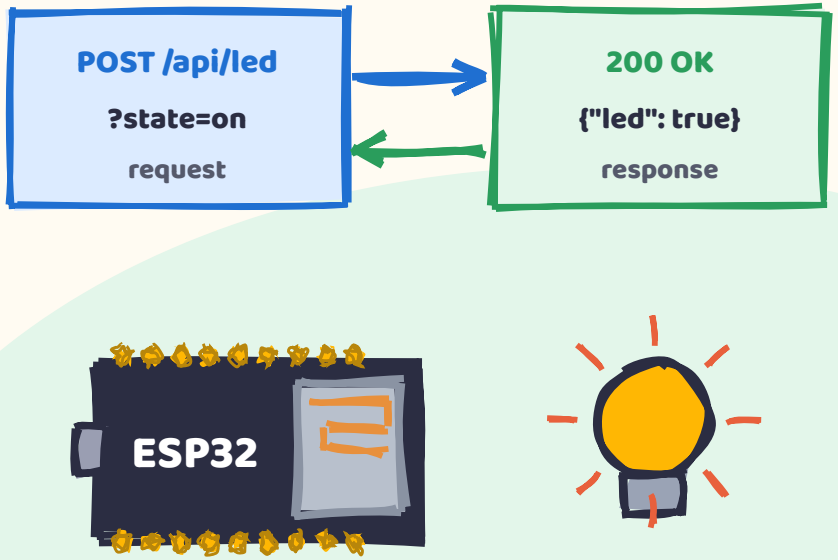
Tech Leaders · Class 12

Web-Controlled Devices via REST API

Make your ESP32 speak the language of the web

HTTP & JSON | ESP32 web server | Web + Python clients | API keys



Did you know?
Almost every app on your phone, from weather to food delivery, gets its data by calling REST APIs.

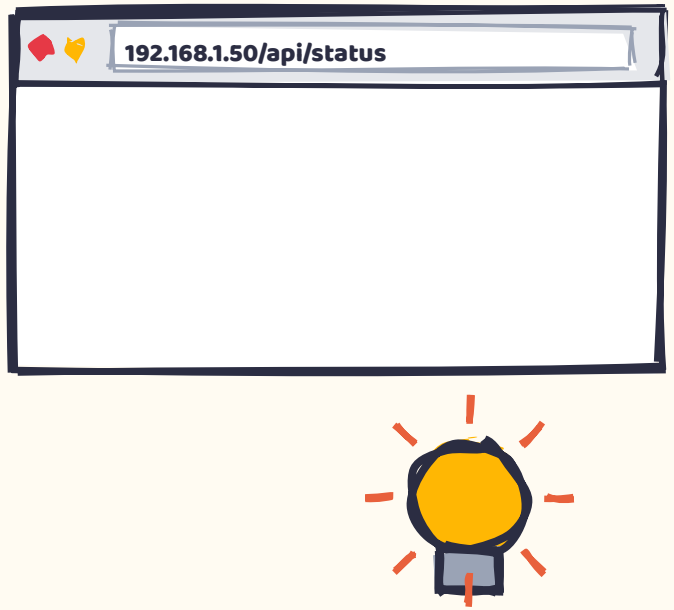
LEARN

Let's think...

How does a phone app switch on a smart light?

Tap → light

You tap a button in an app and a light across the house switches on. The app and the light were made by different people, in different languages. How do they understand each other? They agree on an **API**: a menu of requests the device will accept.



Magic?

Bluetooth?

An API!

Today: you build a **REST API** on the ESP32 and control a lamp from a web page and a Python script.

Learning objectives

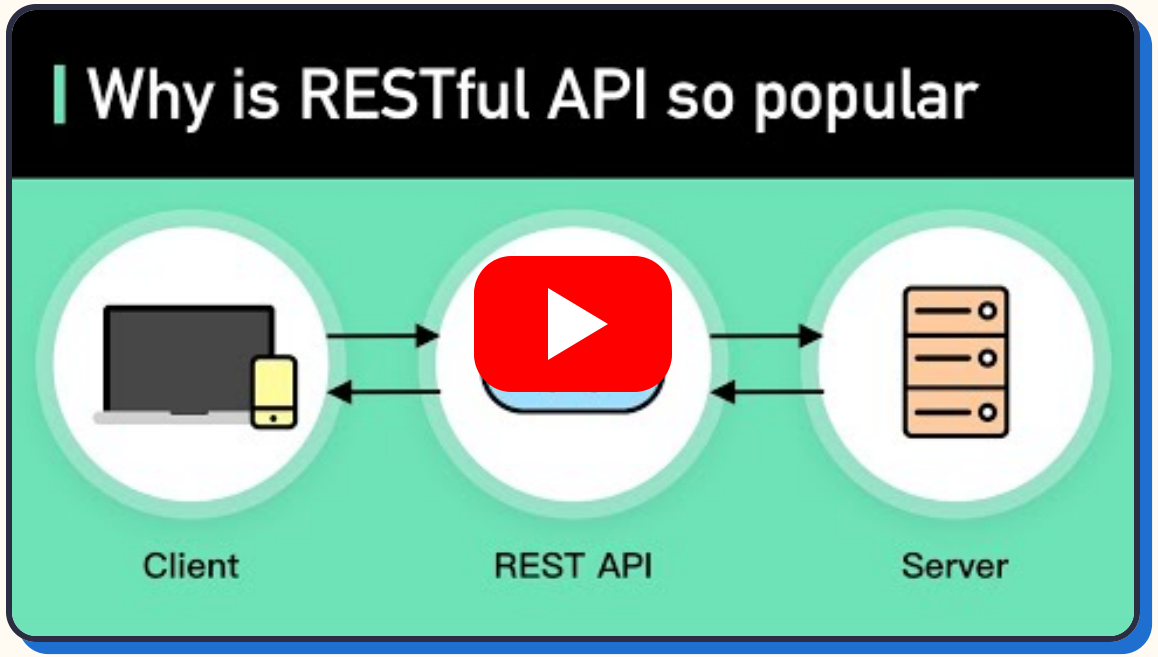
By the end of this class I can...

1	**REST basics:** explain **resources, HTTP methods, status codes and JSON**	**Proof:** label a request and response with method, URL, code and body
2	**API server:** build a **REST API on the ESP32** to read sensors and switch outputs	**Proof:** GET /api/status returns live JSON in a browser
3	**Clients:** control the device from a **web page** and a **Python script**	**Proof:** both clients switch the lamp on and off
4	**Errors:** return the right **error codes** for bad requests	**Proof:** a missing parameter gives 400, a wrong URL gives 404
5	**Secure it:** add an **API key** and explain why devices must not be open to the internet	**Proof:** a request without the key gets 401

LEARN

Objective 1 · REST basics · video

Video time!



What Is REST API? Examples And How To Use It

ByteByteGo · about 5 minutes · youtu.be/-mN3VyJuCjM

Watch for…

1. What is a **resource** in REST?
2. What are GET, POST, PUT and DELETE used for?
3. What do status codes like **200**, **404** and **500** mean?

Scan to watch

After the video: Pick an app on your phone. Guess 2 API requests it makes when you open it (method + URL).

LEARN

Objective 1 · REST basics

Request in, response out

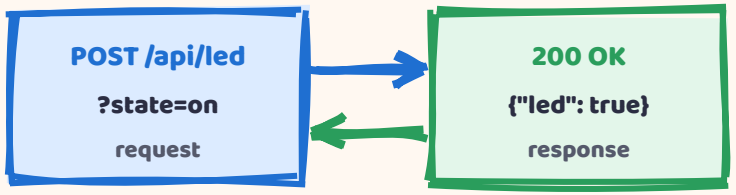
The idea

A **request** has: a **method** (GET = read, POST = do/change), a **URL** naming the resource (**/api/led**), and optional data (**?state=on**).

A **response** has: a **status code** (200 OK, 400 bad request, 401 not allowed, 404 not found) and a **body**, usually **JSON**: **{"led": true, "temp": 24.1}**.

Stateless
REST APIs are 'stateless': every request carries everything the server needs. No memory of earlier chats.

Example: ordering at a restaurant



The menu is the API: it lists what you can ask for. You order (request). The waiter brings food (200 OK) or says 'we don't have that' (404). You don't need to know how the kitchen works.

Real world

APIs connect everything



Weather apps
GET a forecast from a weather service API



Ride and delivery apps
POST an order, GET the driver's location



Smart homes
Google Home and Alexa call device APIs



Payments
Shops call a payment API: never seeing your card number

What could go wrong if a smart lock's API needed no password?

Public vs private
Some APIs are public (weather). Device APIs should be private to you.

LEARN

Objective 3 · Clients · web client

A web page client with fetch()

```
<button onclick="setLamp('on')">Lamp ON</button>
<button onclick="setLamp('off')">Lamp OFF</button>
<p id="out">...</p>
<script>
const ESP = "";       // empty = same device that served this page
async function refresh() {
  const s = await (await fetch(ESP + "/api/status")).json();
  out.textContent = s.temp + " °C, " + s.humidity + " %, lamp " + (s.lamp ? "ON" : "OFF");
}
async function setLamp(state) {
  await fetch(ESP + "/api/lamp?state=" + state, { method: "POST" });
  refresh();
}
setInterval(refresh, 3000);  refresh();
</script>
```

Serve it

Put this HTML in a string and add **server.on("/", ...)** to send it. Now **http://YOUR-IP/** is a control panel.

async / await

fetch() takes time. **await** waits for the answer without freezing the page.

Phones too

Any phone on the same Wi-Fi can open the page. No app store needed!

Objective 5 · Secure it

Lock the API: keys and networks

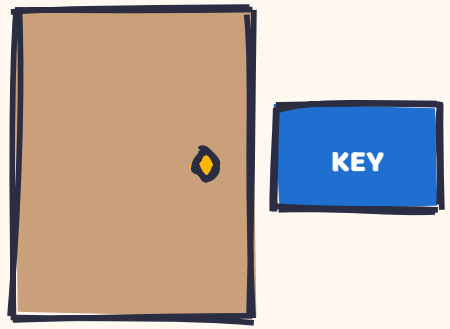
The idea

Right now **anyone on the Wi-Fi** can switch your lamp. Add a simple **API key**: every request must send a header **X-API-Key: our-secret**, otherwise reply **401**.

if (server.header("X-API-Key") != KEY) { server.send(401, ...); return; }
(call **server.collectHeaders()** in setup so the ESP32 keeps that header)

And never 'port-forward' a home device to the open internet. Bots scan for them constantly.

Example: a hotel key card



The hotel door is on a public corridor, but only your card opens it. An API key is a card that every request must show. Lose the card, and you change the lock (the key).

Not perfect
An API key over plain HTTP can be sniffed on the same network. Real products use HTTPS too.

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Human API
2. Our API kit (per group)
3. Wire the device
4. ESP32 API part 1: Wi-Fi and routes
5. ESP32 API part 2: handlers and errors
6. Test like a developer
7. A Python client that automates

Class game

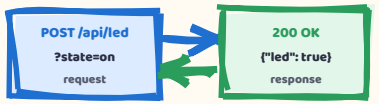
Human API

How to play

1. One student is the **ESP32 server** with a lamp (torch) and a thermometer.
2. Others are **clients**. They pass written request cards: **GET /api/status**, **POST /api/led?state=on**…
3. The server may only reply with a status code and a JSON card.
4. Teacher sneaks in bad cards: **POST /api/led** (no state), **GET /api/pizza**.

Then discuss

Which status code did you give each bad card? Why do clear error codes help the programmer on the other side?



Big idea
An API is a contract: exact requests, exact replies, clear errors.

MAKE

Build it

Our API kit (per group)



ESP32 + USB
Runs the web server



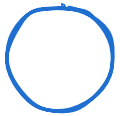
1-channel relay
Switches the 5 V lamp



5 V LED lamp
The device we control



DHT22
Temperature and humidity



2 LEDs + 220 Ω
Status lights



Laptop
Browser, Python, curl

Laptop and ESP32 must be on the **same Wi-Fi network**. School networks sometimes block devices from seeing each other: use the teacher's lab router.

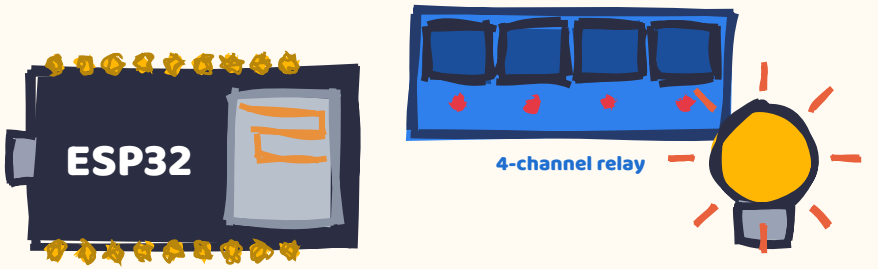
Low voltage ONLY
The relay switches the 5 V lamp only. NEVER connect mains (220 V) to a breadboard relay in class.

MAKE

Objective 2 · API server · Build step 1 of 4

Wire the device

Part	Connect to
Relay IN / VCC / GND	GPIO 23 / 5V (VIN) / GND
Lamp +	5V → relay COM, relay NO → lamp +
Lamp –	GND
Status LED + 220 Ω	GPIO 18
DHT22 DATA / VCC / GND	GPIO 4 / 3V3 / GND



NO and COM

NO = normally open: the lamp is off until the relay clicks. If your lamp is on when it should be off, you used **NC**.

Objective 2 · API server · Build step 2 of 4

ESP32 API part 1: Wi-Fi and routes

```
#include <WiFi.h>
#include <WebServer.h>
#include "DHT.h"
const char* SSID = "RoboticsLab";
const char* PASS = "********";
WebServer server(80);
DHT dht(4, DHT22);
const int RELAY = 23, LED = 18;
bool lamp = false;

void setup() {
  Serial.begin(115200);  dht.begin();
  pinMode(RELAY, OUTPUT);  pinMode(LED, OUTPUT);
  WiFi.begin(SSID, PASS);
  while (WiFi.status() != WL_CONNECTED) delay(300);
  Serial.println(WiFi.localIP());          // write this down!

  server.on("/api/status", HTTP_GET,  sendStatus);
  server.on("/api/lamp",   HTTP_POST, setLamp);
  server.onNotFound(notFound);
  server.begin();
}
void loop() { server.handleClient(); }
```

Routes

server.on(URL, METHOD, function) connects each request to the function that answers it.

IP address

The Serial Monitor prints something like **192.168.1.50**. That is your device's address.

Order

In the Arduino IDE, put the three handler functions (next slide) **above** setup().

Objective 4 · Errors · Build step 2 of 4

ESP32 API part 2: handlers and errors

```
void sendStatus() {
  String j = "{\"temp\":" + String(dht.readTemperature(), 1) +
             ",\"humidity\":" + String(dht.readHumidity(), 1) +
             ",\"lamp\":" + (lamp ? "true" : "false") + "}";
  server.send(200, "application/json", j);
}

void setLamp() {
  if (!server.hasArg("state")) {            // bad request
    server.send(400, "application/json", "{\"error\":\"state missing\"}");
    return;
  }
  lamp = server.arg("state") == "on";
  digitalWrite(RELAY, lamp);
  digitalWrite(LED, lamp);
  sendStatus();                             // reply with the new state
}

void notFound() {
  server.send(404, "application/json", "{\"error\":\"not found\"}");
}
```

Test

Open **http://YOUR-IP/api/status** in a browser. You should see live JSON.

\"?

Inside a C string, **\"** means a quote character. JSON needs lots of them!

Relay backwards?

Some relay boards switch on LOW. If so, write **!lamp** to the relay pin.

MAKE

Objective 3 · Clients · Build step 3 of 4

Test like a developer

```
# read (GET is the browser default)
curl http://192.168.1.50/api/status

# change something (POST)
curl -X POST "http://192.168.1.50/api/lamp?state=on"
curl -X POST "http://192.168.1.50/api/lamp?state=off"

# break it on purpose
curl -i -X POST http://192.168.1.50/api/lamp      # 400?
curl -i http://192.168.1.50/api/pizza             # 404?
```

curl

A command-line client built into Windows, Mac and Linux. **-X** picks the method, **-i** shows the status code.

Why POST to change?

Browsers, link previews and search bots freely **GET** URLs. A GET that switches things on could be triggered by accident.

Objective 3 · Clients · Build step 4 of 4

A Python client that automates

```python
import requests, time

ESP = "http://192.168.1.50"          # your ESP32's IP

status = requests.get(ESP + "/api/status", timeout=3).json()
print("Temperature:", status["temp"], "°C")

# automation: lamp on when it's warm, off when cool
while True:
    s = requests.get(ESP + "/api/status", timeout=3).json()
    want = "on" if s["temp"] > 27 else "off"
    r = requests.post(ESP + "/api/lamp", params={"state": want}, timeout=3)
    print(time.strftime("%H:%M:%S"), s["temp"], "->", want, r.status_code)
    time.sleep(10)
```

.json()

Turns the JSON text into a Python **dictionary**, so **s["temp"]** just works.

timeout

Always set one. Without it, a sleeping ESP32 freezes your script forever.

Any language

The same API works from Python, JavaScript, an app or another ESP32. That is the power of REST.

MAKE

Test like an engineer

API test sheet

Request	Expected	We got
GET /api/status	200 + JSON	
POST /api/lamp?state=on	200, lamp on	
POST /api/lamp (no state)	400	
GET /api/pizza	404	
Any request without the key	401	
Web page and Python both work	✓	

Debug checklist

Browser can't connect? Wrong IP, different network, or school Wi-Fi blocks device-to-device traffic.
Temp shows nan? Check the DHT22 wiring.

Write the expected result BEFORE you test. That is how professional testers catch surprises.

Design it well

Good API habits

Habit	Example
Nouns in URLs	/api/lamp not /api/turnLampOn
Methods for actions	GET reads, POST/PUT change, DELETE removes
Honest status codes	never 200 with an error inside
Version it	/api/v1/... so old clients keep working
Document it	a table of every route, its inputs and outputs

Your turn

Write a one-page API document for your device, as if another team will build the app.

MAKE

Level up!

API challenges

Choose your challenge



Timer: POST /api/lamp?state=on&minutes=5 switches off automatically.



History: GET /api/history returns the last 20 readings as a JSON array.



App: build an MIT App Inventor app using the Web component.



Interoperate: control ANOTHER team's device using only their API document.

Think like an engineer

What should your API do if two people send 'on' and 'off' at the same moment?

Boss level

Add HTTPS-style thinking: rate-limit to 5 requests per second and log every client IP.

Objective check

Can I really do it? Prove it.

01	Label a request and response: method, URL, status code, body.	○ ○ ○
02	Open /api/status in a browser and read the JSON.	○ ○ ○
03	Switch the lamp from your web page AND your Python script.	○ ○ ○
04	Show a 400 and a 404. What did you send to cause each?	○ ○ ○
05	Show a request being refused with 401. Why shouldn't this device be on the open internet?	○ ○ ○

Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: Integration test: swap API documents with another team and control their lamp from your Python script.

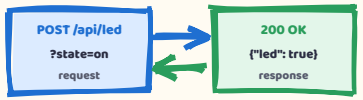
Show what you know

What did we learn?

- [x] An API is a contract between programs
- [x] Request = method + URL + data; response = status code + JSON
- [x] The ESP32 WebServer maps routes to functions
- [x] Any client (browser, Python, app) can use the same API
- [x] Protect devices with keys and keep them off the open internet

At home

Open any website, press F12 → Network, and watch the API calls it makes. Screenshot one JSON response.



Exit ticket: a client sends POST /api/lamp with no state. What should the ESP32 reply, and why?

Next class: Securing an IoT Device!